Describe how to load data and make it available to Keras?

To load data and make it available to Keras, we first import the **NumPy** library, which is used to handle numerical arrays. The dataset file (`pima-indians-diabetes.csv`) is stored locally in the same directory as the Python script. We use the NumPy function `loadtxt()` to read the CSV file and convert it into a two-dimensional array of numbers.

```
from numpy import loadtxt
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
```

After loading, we separate the **input features (X)** and the **output labels (y)** using array slicing.

```
X = dataset[:, 0:8]   # first 8 columns (features)
y = dataset[:, 8]     # last column (target)
```

Keras models require numerical input arrays (`X`) and target labels (`y`), so once the dataset is loaded and split like this, it is ready to be passed to the model's training function `model.fit(X, y, ...)`.

Describe the dataset used in the tutorial: what does label 1 mean in the actual data? What does label 0 mean in the actual data?

The dataset used in this tutorial is the **Pima Indians Diabetes Dataset**, a well-known dataset from the **UCI Machine Learning Repository**.
It contains **medical diagnostic measurements** collected from female Pima Indian patients who are at least 21 years old. The goal is to predict whether each patient is likely to develop diabetes within five years.

Each row represents one patient and includes **8 numerical input features** (such as glucose level, blood pressure, BMI, etc.) and **1 output label** that indicates the diagnosis result.

- **Label 1** → The patient **had an onset of diabetes** within five years.
- **Label 0** → The patient **did not have diabetes** within that period.

Thus, the dataset is a **binary classification problem**, where the neural network learns to map medical indicators (X) to a diabetes outcome (y = 0 or 1).

How many features are in the dataset? What are the features?

The Pima Indians Diabetes dataset contains **8 input features** (also called predictor variables). Each feature represents a measurable medical attribute of a patient that could help predict the likelihood of diabetes.

Here are the eight features:

1. **Number of times pregnant**
2. **Plasma glucose concentration** measured 2 hours after an oral glucose tolerance test
3. **Diastolic blood pressure** (in mm Hg)
4. **Triceps skinfold thickness** (in mm)
5. **2-hour serum insulin** (in μU/ml)
6. **Body Mass Index (BMI)** = weight in kg / (height in m)^2
7. **Diabetes pedigree function**, which estimates the genetic likelihood of diabetes
8. **Age** (in years)

These features together form the **input vector X**, which is passed to the neural network to predict the output label (0 or 1).

Describe how to create a neural network model with Keras. How many layers are used? How many nodes are used in each layer?

To create a neural network model with Keras, we first import the necessary classes from the `keras.models` and `keras.layers` modules. We then define a Sequential model, which allows us to build the network layer by layer in order.

```
from keras.models import Sequential
from keras.layers import Dense
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))   # first hidden layer
model.add(Dense(8, activation='relu'))              # second hidden layer
model.add(Dense(1, activation='sigmoid'))
```

- **Model type:** Sequential (each layer feeds directly into the next)
- **Number of used layers:** 3 layers (2 hidden layers + 1 output layer)
- **Nodes per layer:**
    - **Layer 1:** 12 neurons (ReLU activation)
    - **Layer 2:** 8 neurons (ReLU activation)
    - **Output Layer:** 1 neuron (Sigmoid activation for binary classification)

In Keras, the input layer is automatically created when you specify the first layer's input size — it doesn't have trainable parameters, so it's not shown as a "layer" in `model.summary()`.

The model expects **8 input features** (`input_dim=8`) and outputs a single probability between **0 and 1**, indicating the likelihood of diabetes.

Explain how the Param # (hyperparameters) are computed, as shown by the model.summary()?

When we call `model.summary()` in Keras, it displays a table that lists each layer, its output shape, and the number of trainable parameters (Param #).
These parameters represent all the weights and biases that the neural network learns during training.

Each Dense (fully connected) layer's parameter count is calculated using the formula:

Parameters=(Number of inputs+1)×Number of neurons in the layer

The "+1" accounts for the bias term associated with each neuron.

Let's compute them for our model:

1. First Hidden Layer:
    - Inputs = 8
    - Neurons = 12
    - Parameters = (8 + 1) × 12 = 108

2. Second Hidden Layer:
    - Inputs = 12
    - Neurons = 8
    - Parameters = (12 + 1) × 8 = 104
3. Output Layer:
    - Inputs = 8
    - Neurons = 1
    - Parameters = (8 + 1) × 1 = 9

Total Trainable Parameters: 108 + 104 + 9 = 221

These values are shown in the `model.summary()` output, confirming how many adjustable weights the network learns to optimize during training.

Describe what a Rectified Linear Unit (ReLU) is, in your own words

A Rectified Linear Unit (ReLU) is an activation function commonly used in neural networks to introduce non-linearity.
It helps the model learn complex patterns instead of just simple linear relationships.

Mathematically, the ReLU function is defined as:

$$f(x)= \max(0,x)$$

This means that for any input x, the output is x if x > 0, and 0 otherwise.

In simple terms, ReLU passes positive values unchanged and replaces negative values with zero.

ReLU is popular because:

- It makes networks train faster (less computationally expensive).
- It helps reduce the “vanishing gradient” problem that older functions like Sigmoid or Tanh often had.

- It allows models to focus on active (positive) neurons while ignoring inactive ones.

Describe how to use Keras to evaluate models.

After training a Keras model with `model.fit()`, we can measure how well it performs by using the **`evaluate()`** function. This function runs the model on a given dataset (usually test data) and returns numerical results for the **loss** and **metrics** (such as accuracy) that were specified during compilation.

```
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy * 100))
```

How it works:

1. The `evaluate()` function takes input data (`X`) and true labels (`y`).
2. It uses the trained model to make predictions.
3. It compares the predictions to the true labels and computes:
    - **Loss value** (e.g., binary cross-entropy)
    - **Metric values** (e.g., accuracy)
4. The function returns these values — in this example, we only display the accuracy as a percentage.

This allows us to **quantitatively measure** how well the model learned to map inputs to outputs. For more advanced experiments, separate training and test datasets should be used to evaluate the model's ability to generalize to unseen data.

```
import numpy as np
import tensorflow as tf
import keras
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
import pandas as pd
```

```
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

```
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
print(model.summary())
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 12)	108
dense_13 (Dense)	(None, 8)	104
dense_14 (Dense)	(None, 1)	9

Total params: 221 (884.00 B)
Trainable params: 221 (884.00 B)
Non-trainable params: 0 (0.00 B)
None

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
model.fit(X, y, epochs=150, batch_size=10)
```

```
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7000 - loss: 0.5635
Epoch 44/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6809 - loss: 0.6005
Epoch 45/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6768 - loss: 0.6115
Epoch 46/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6801 - loss: 0.6102
Epoch 47/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6796 - loss: 0.5566
Epoch 48/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6941 - loss: 0.5658
Epoch 49/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7220 - loss: 0.5658
Epoch 50/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7097 - loss: 0.5492
Epoch 51/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6786 - loss: 0.5883
Epoch 52/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6784 - loss: 0.5839
Epoch 53/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7156 - loss: 0.5604
Epoch 54/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6863 - loss: 0.5883
Epoch 55/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7194 - loss: 0.5497
Epoch 56/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7091 - loss: 0.5651
Epoch 57/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6954 - loss: 0.5681
Epoch 58/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6558 - loss: 0.6053
Epoch 59/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7324 - loss: 0.5361
Epoch 60/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7244 - loss: 0.5504
Epoch 61/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6906 - loss: 0.5873
Epoch 62/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.6996 - loss: 0.5827
Epoch 63/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7133 - loss: 0.5683
Epoch 64/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7279 - loss: 0.5604
Epoch 65/150
77/77 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7312 - loss: 0.5650
```

```
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))
```

```
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7337 - loss: 0.5421
Accuracy: 75.00
```

```
df = pd.DataFrame(dataset)
```

```
df
```

	0	1	2	3	4	5	6	7	8
0	6.0	148.0	72.0	35.0	0.0	33.6	0.627	50.0	1.0
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351	31.0	0.0
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672	32.0	1.0
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167	21.0	0.0
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288	33.0	1.0
...	...	...	...	...	...	...	...	...	...
763	10.0	101.0	76.0	48.0	180.0	32.9	0.171	63.0	0.0
764	2.0	122.0	70.0	27.0	0.0	36.8	0.340	27.0	0.0
765	5.0	121.0	72.0	23.0	112.0	26.2	0.245	30.0	0.0
766	1.0	126.0	60.0	0.0	0.0	30.1	0.349	47.0	1.0
767	1.0	93.0	70.0	31.0	0.0	30.4	0.315	23.0	0.0

768 rows × 9 columns